

Vietnam National University - HCMC University of Science

Faculty of Information Technology



Software Design

Course: Introduction to Software Engineering

Students that participated in this project:

Đinh Ngọc Anh Dương – 23127039

Trần Thị Xuân Tài – 23127256

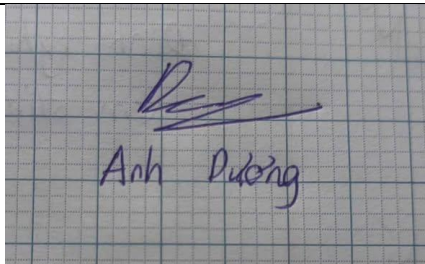
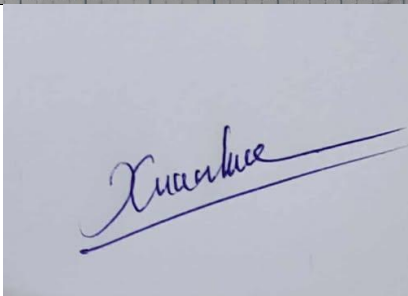
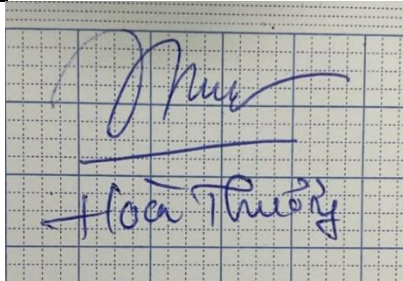
Trần Thọ - 23127264

Nguyễn Tuấn An – 23127316

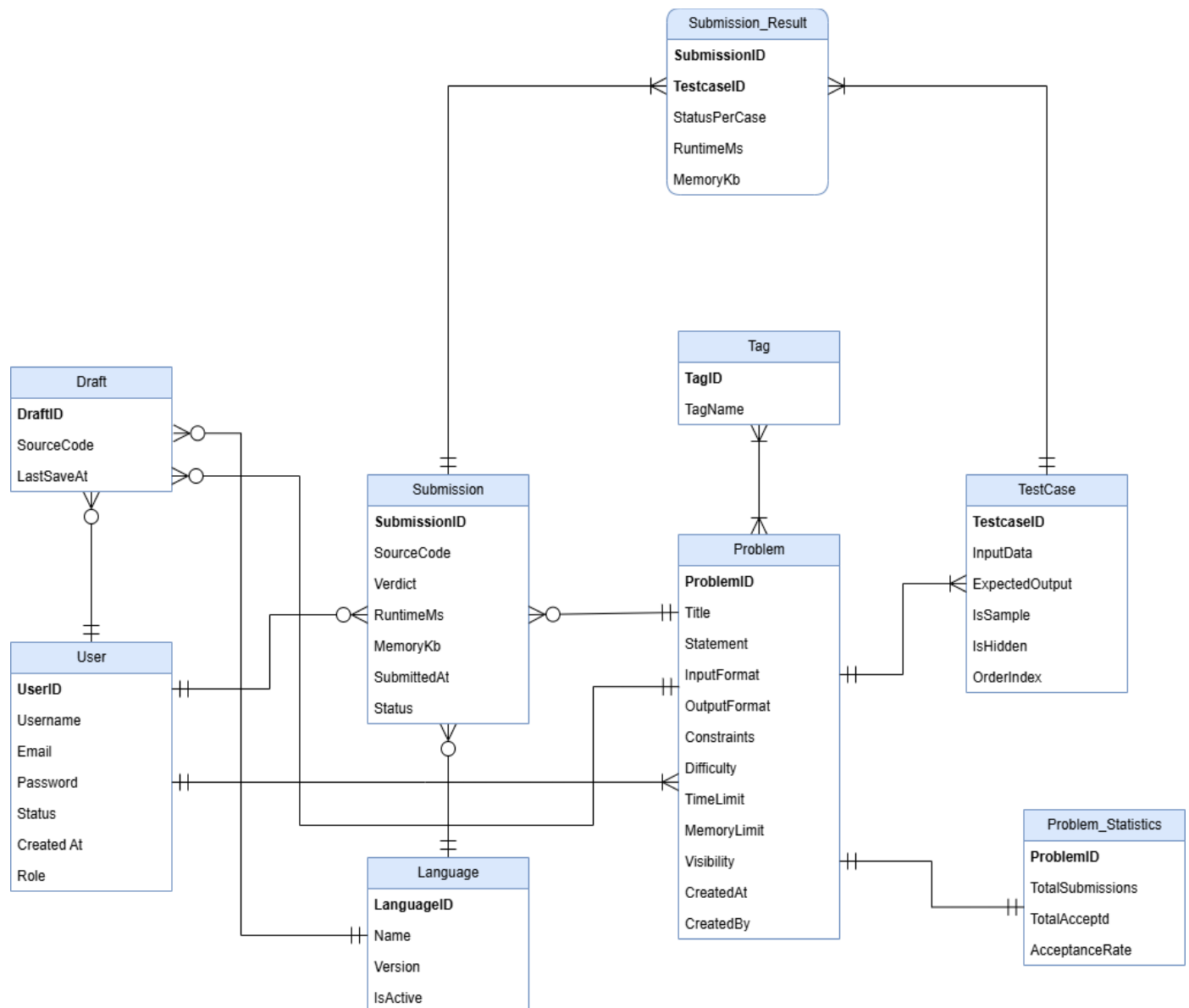
Đỗ Thị Hoài Thương - 23127492

Instructor: Trương Phước Lộc

1.Member Contribution Assessment

ID	Name	Contribution(%)	Signature
23127039	Đinh Ngọc Anh Dương	100%	
23127256	Trần Thị Xuân Tài	100%	
23127264	Trần Thọ	100%	
23127316	Nguyễn Tuấn An	100%	
23127492	Đỗ Thị Hoài Thương	100%	

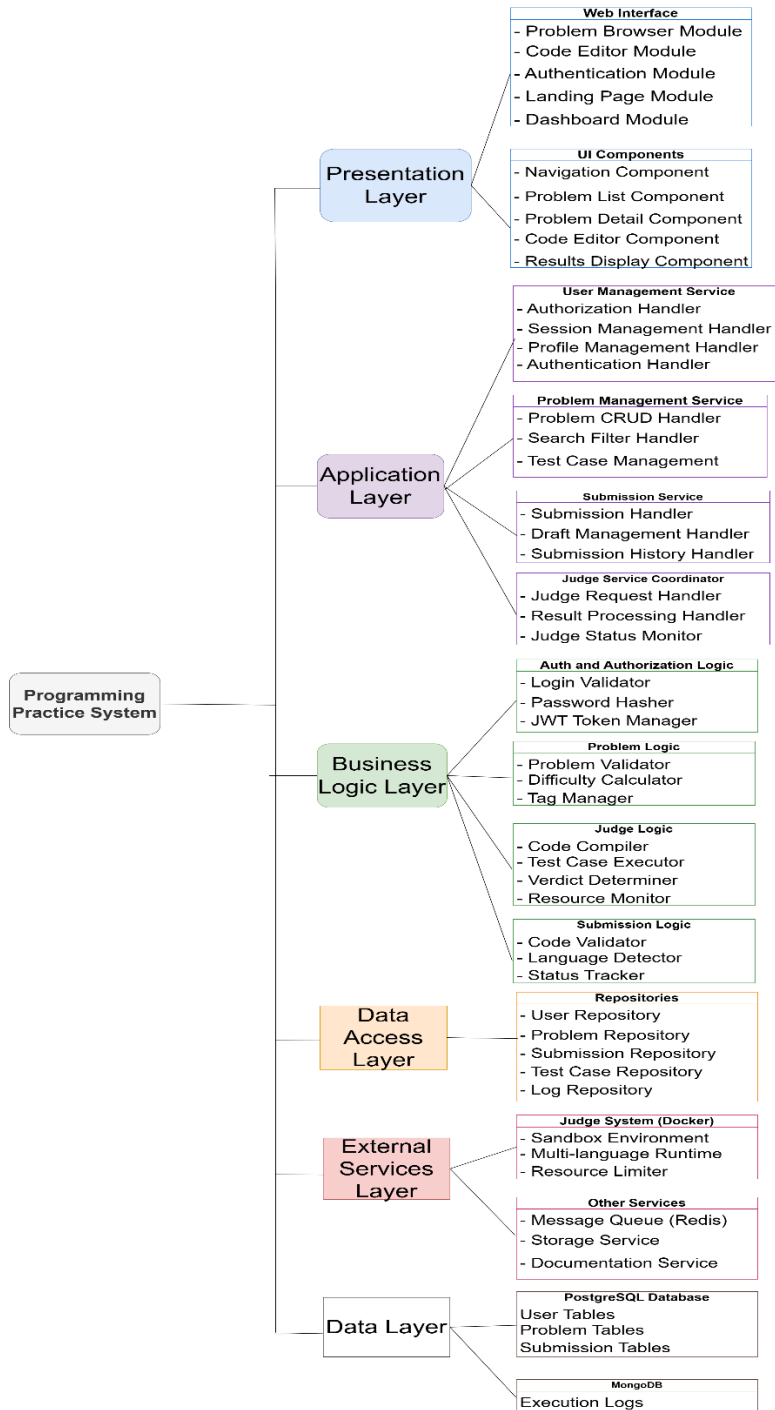
2. Conceptual Model



3. Architectural Design

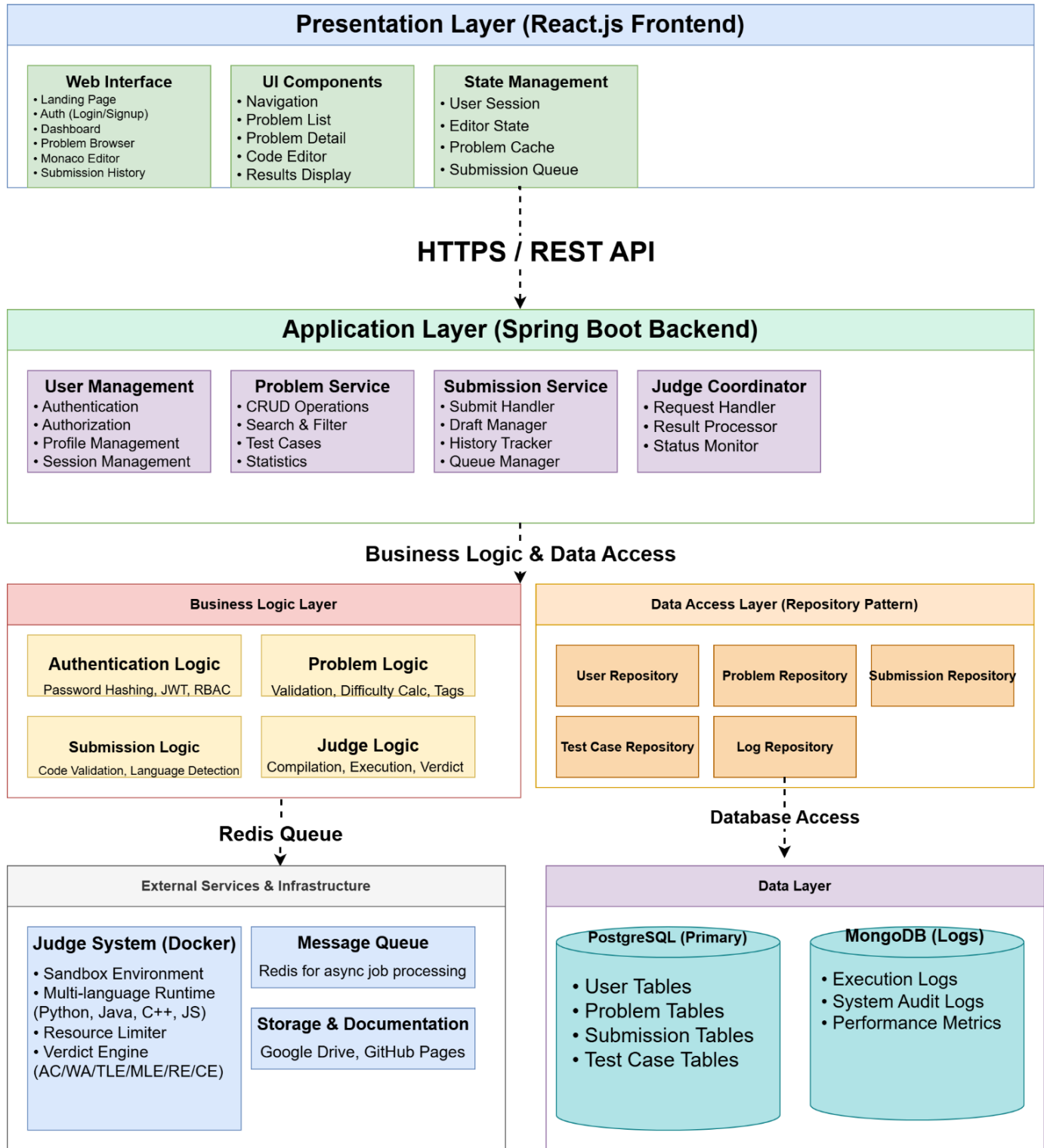
3.1. Architecture Diagram

3.1.1. System decomposition tree diagram:

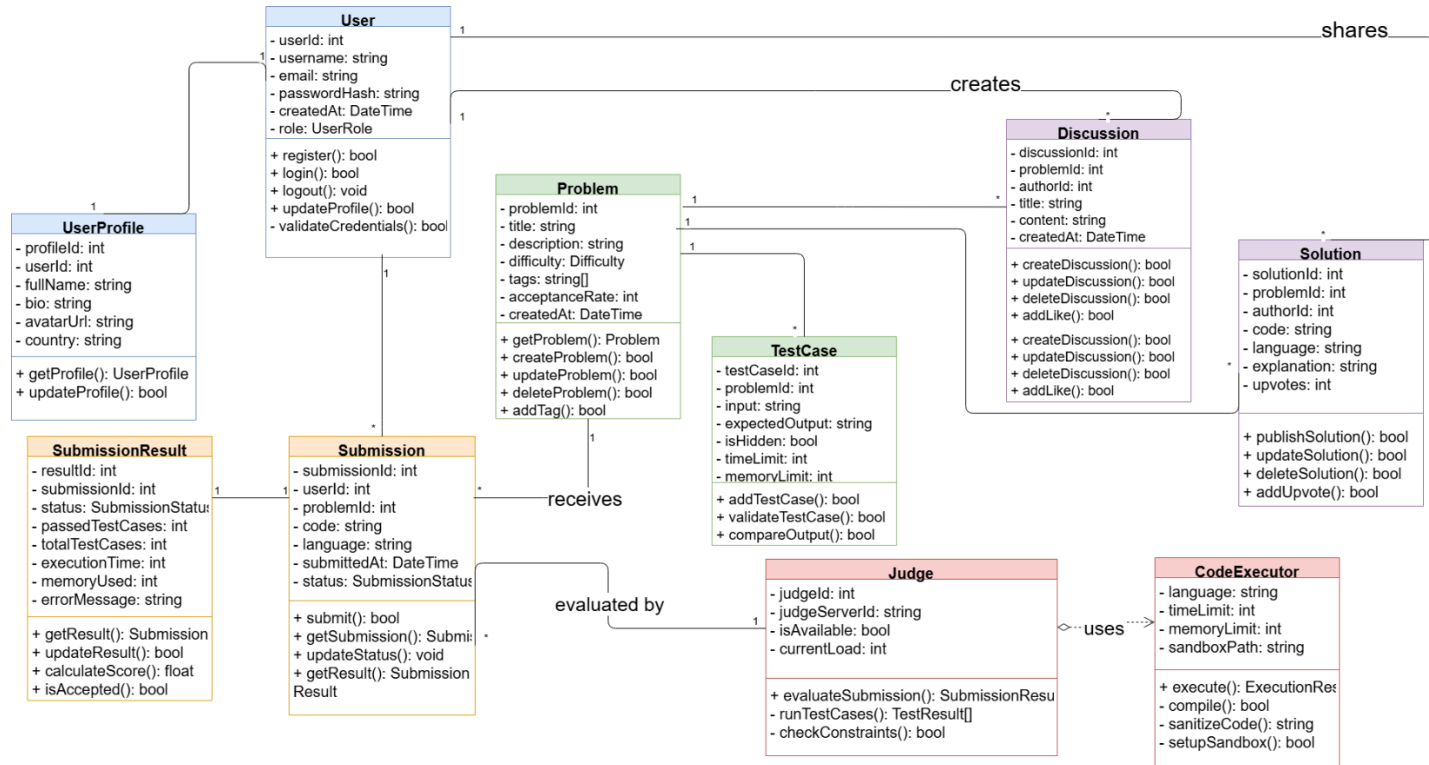


3.1.2. Overall System Architecture

Programming Practice System - Architecture



3.2. Class Diagram



3.3. Class Specifications

3.3.1. Class: User

- **Inheritance:** None (Base class)
- **Description:** Represents a user account in the system. Handles authentication and basic user information.
- **Attributes:**

Seq	Property	Modifier	Constraint	Description
1	userId	private	Primary Key, Auto-increment	Unique identifier for the user
2	username	private	Unique, NOT NULL, 3-30 chars	User's login name
3	email	private	Unique, NOT NULL, Valid email format	User's email address
4	passwordHash	private	NOT NULL, Min 60 chars (bcrypt)	Encrypted password
5	createdAt	private	NOT NULL, Default: Current timestamp	Account creation date
6	role	private	NOT NULL, Default: USER	User role (USER, ADMIN, MODERATOR)

- **Operations:**

Seq	Operation	Modifier	Constraint	Description
1	register(username, email, password)	public	Returns bool	Creates a new user account

2	login(username, password)	public	Returns bool	Authenticates user credentials
3	logout()	public	Returns void	Ends user session
4	updateProfile(profileData)	public	Returns bool	Updates user profile information
5	validateCredentials(password)	private	Returns bool	Validates password against hash
6	changePassword(oldPassword, newPassword)	public	Returns bool	Changes user password

3.3.2. Class: Problem

- **Inheritance:** None (Base class)
- **Description:** Represents a programming problem/challenge in the system.
- **Attributes:**

Seq	Property	Modifi er	Constraint	Description
1	problemId	private	Primary Key, Auto-increment	Unique identifier for the problem
2	title	private	NOT NULL, Max 200 chars	Problem title
3	description	private	NOT NULL, Text	Detailed problem description with examples

4	difficulty	private	NOT NULL, Enum (EASY, MEDIUM, HARD)	Problem difficulty level
5	tags	private	Array of strings	Topic tags (e.g., Array, DFS, DP)
6	acceptanceRate	private	Float, 0-100	Percentage of accepted submissions
7	createdAt	private	NOT NULL, Timestamp	Problem creation date
8	authorId	private	Foreign Key	ID of problem creator

- Operations:

Seq	Operation	Modifier	Constraint	Description
1	getProblem(problemId)	public static	Returns Problem	Retrieves problem by ID
2	createProblem(problemData)	public	Returns bool	Creates a new problem
3	updateProblem(problemData)	public	Returns bool	Updates problem details
4	deleteProblem()	public	Returns bool	Soft deletes the problem
5	addTag(tag)	public	Returns bool	Adds a topic tag
6	updateAcceptanceRate()	private	Returns void	Recalculates acceptance rate

3.3.3. Class: TestCase

- Inheritance: None

- **Description:** Represents a test case for validating submissions against a problem.
- **Attributes:**

Seq	Property	Modifier	Constraint	Description
1	testCaseId	private	Primary Key, Auto-increment	Unique identifier for test case
2	problemId	private	Foreign Key, NOT NULL	Associated problem ID
3	input	private	NOT NULL, Text	Input data for the test
4	expectedOutput	private	NOT NULL, Text	Expected output result
5	isHidden	private	Boolean, Default: false	Whether test case is visible to users
6	timeLimit	private	Integer, Milliseconds	Maximum execution time allowed
7	memoryLimit	private	Integer, MB	Maximum memory allowed

- **Operations:**

Seq	Operation	Modifier	Constraint	Description
1	addTestCase(problemId, input, output)	public	Returns bool	Adds a new test case
2	validateTestCase()	public	Returns bool	Validates test case format
3	updateTestCase(testCaseData)	public	Returns bool	Updates test case details

4	deleteTestCase()	public	Returns bool	Removes test case
5	compareOutput(actualOutput)	public	Returns bool	Compares actual vs expected output

3.3.4. Class: Submission

- **Inheritance:** None
- **Description:** Represents a code submission from a user for a specific problem.
- **Attributes:**

Seq	Property	Modifier	Constraint	Description
1	submissionId	private	Primary Key, Auto-increment	Unique identifier for submission
2	userId	private	Foreign Key, NOT NULL	ID of user who submitted
3	problemId	private	Foreign Key, NOT NULL	ID of problem being solved
4	code	private	NOT NULL, Text	Source code submitted
5	language	private	NOT NULL, Enum	Programming language used
6	submittedAt	private	NOT NULL, Timestamp	Submission timestamp
7	status	private	Enum, Default: PENDING	Current status of submission

- **Operations:**

Seq	Operation	Modifier	Constraint	Description
1	submit(userId, problemId, code)	public	Returns bool	Submits code for evaluation
2	getSubmission(submissionId)	public static	Returns Submission	Retrieves submission details
3	updateStatus(status)	public	Returns void	Updates submission status
4	getCode()	public	Returns string	Returns submitted code
5	getResult()	public	Returns SubmissionResult	Gets evaluation result

3.3.5. Class: SubmissionResult

- **Inheritance:** None
- **Description:** Stores the evaluation results of a code submission.
- **Attributes:**

Seq	Property	Modifier	Constraint	Description
1	resultId	private	Primary Key, Auto-increment	Unique identifier for result
2	submissionId	private	Foreign Key, Unique, NOT NULL	Associated submission ID
3	status	private	Enum, NOT NULL	Final verdict

4	passedTestCases	private	Integer, ≥ 0	Number of test cases passed
5	totalTestCases	private	Integer, > 0	Total number of test cases
6	executionTime	private	Integer, Milliseconds	Total execution time
7	memoryUsed	private	Integer, KB	Memory consumed
8	errorMessage	private	Text, Nullable	Error details if failed

- Operations:

Seq	Operation	Modifier	Constraint	Description
1	getResult(submissionId)	public static	Returns SubmissionResult	Gets result by submission ID
2	updateResult(resultData)	public	Returns bool	Updates result information
3	calculateScore()	public	Returns float	Calculates score based on results
4	isAccepted()	public	Returns bool	Checks if submission passed

3.3.6. Class: Judge

- **Inheritance:** None
- **Description:** Handles the evaluation and judging of code submissions.

- Attributes:

Seq	Property	Modifier	Constraint	Description
1	judgeId	private	Primary Key, Auto-increment	Unique identifier for judge instance
2	judgeServerId	private	NOT NULL	Server ID handling this judge
3	isAvailable	private	Boolean, Default: true	Whether judge is available
4	currentLoad	private	Integer, ≥ 0	Number of active evaluations

- Operations:

Seq	Operation	Modifier	Constraint	Description
1	evaluateSubmission(submission)	public	Returns SubmissionResult	Evaluates a complete submission
2	runTestCases(code, testCases)	private	Returns TestResult[]	Runs code against test cases
3	checkConstraints(result)	private	Returns bool	Validates time/memory constraints
4	assignToServer()	private	Returns string	Assigns to available server

3.3.7. Class: CodeExecutor

- **Inheritance:** None
- **Description:** Executes user code in a sandboxed environment.
- **Attributes:**

Seq	Property	Modifier	Constraint	Description
1	language	private	NOT NULL	Programming language
2	timeLimit	private	Integer, Milliseconds	Maximum execution time
3	memoryLimit	private	Integer, MB	Maximum memory usage
4	sandboxPath	private	String	Path to isolated execution environment

- **Operations:**

Seq	Operation	Modifier	Constraint	Description
1	execute(code, input)	public	Returns ExecutionResult	Executes code with input
2	compile(code)	private	Returns bool	Compiles code (if needed)
3	sanitizeCode(code)	private	Returns string	Removes dangerous code patterns
4	setupSandbox()	private	Returns bool	Prepares isolated environment
5	cleanup()	private	Returns void	Cleans up after execution

3.3.8. Class: Discussion

- **Inheritance:** None
- **Description:** Represents a discussion thread related to a problem.
- **Attributes:**

Seq	Property	Modifier	Constraint	Description
1	discussionId	private	Primary Key, Auto-increment	Unique identifier for discussion
2	problemId	private	Foreign Key, NOT NULL	Associated problem ID
3	authorId	private	Foreign Key, NOT NULL	ID of discussion creator
4	title	private	NOT NULL, Max 200 chars	Discussion title
5	content	private	NOT NULL, Text	Discussion content/description
6	createdAt	private	NOT NULL, Timestamp	Creation timestamp
7	updatedAt	private	Timestamp	Last update timestamp
8	likes	private	Integer, >= 0	Number of likes
9	isPinned	private	Boolean, Default: false	Whether discussion is pinned

- **Operations:**

Seq	Operation	Modifier	Constraint	Description
-----	-----------	----------	------------	-------------

1	createDiscussion (problemId, title, content)	public	Returns bool	Creates new discussion
2	updateDiscussion(discussionData)	public	Returns bool	Updates discussion details
3	deleteDiscussion ()	public	Returns bool	Deletes discussion thread
4	addLike(userId)	public	Returns bool	Adds a like from user
5	removeLike(userId)	public	Returns bool	Removes a like

3.3.9. Class: Solution

- **Inheritance:** None
- **Description:** Represents a user-contributed solution to a problem.
- **Attributes:**

Seq	Property	Modifier	Constraint	Description
1	solutionId	private	Primary Key, Auto-increment	Unique identifier for solution
2	problemId	private	Foreign Key, NOT NULL	Associated problem ID
3	authorId	private	Foreign Key, NOT NULL	ID of solution author
4	code	private	NOT NULL, Text	Solution code
5	language	private	NOT NULL	Programming language

6	explanation	private	Text	Detailed explanation of approach
7	upvotes	private	Integer, >= 0	Number of upvotes
8	createdAt	private	NOT NULL, Timestamp	Publication date
9	isOfficial	private	Boolean, Default: false	Whether it's an official solution

- Operations:

Seq	Operation	Modifier	Constraint	Description
1	publishSolution (problemId, code, explanation)	public	Returns bool	Publishes a new solution
2	updateSolution(solutionData)	public	Returns bool	Updates solution details
3	deleteSolution ()	public	Returns bool	Removes solution
4	addUpvote(userId)	public	Returns bool	Adds an upvote
5	removeUpvote(userId)	public	Returns bool	Removes an upvote

3.3.10. Class: UserStatistics

- **Inheritance:** None
- **Description:** Tracks and maintains user performance statistics.

- Attributes:

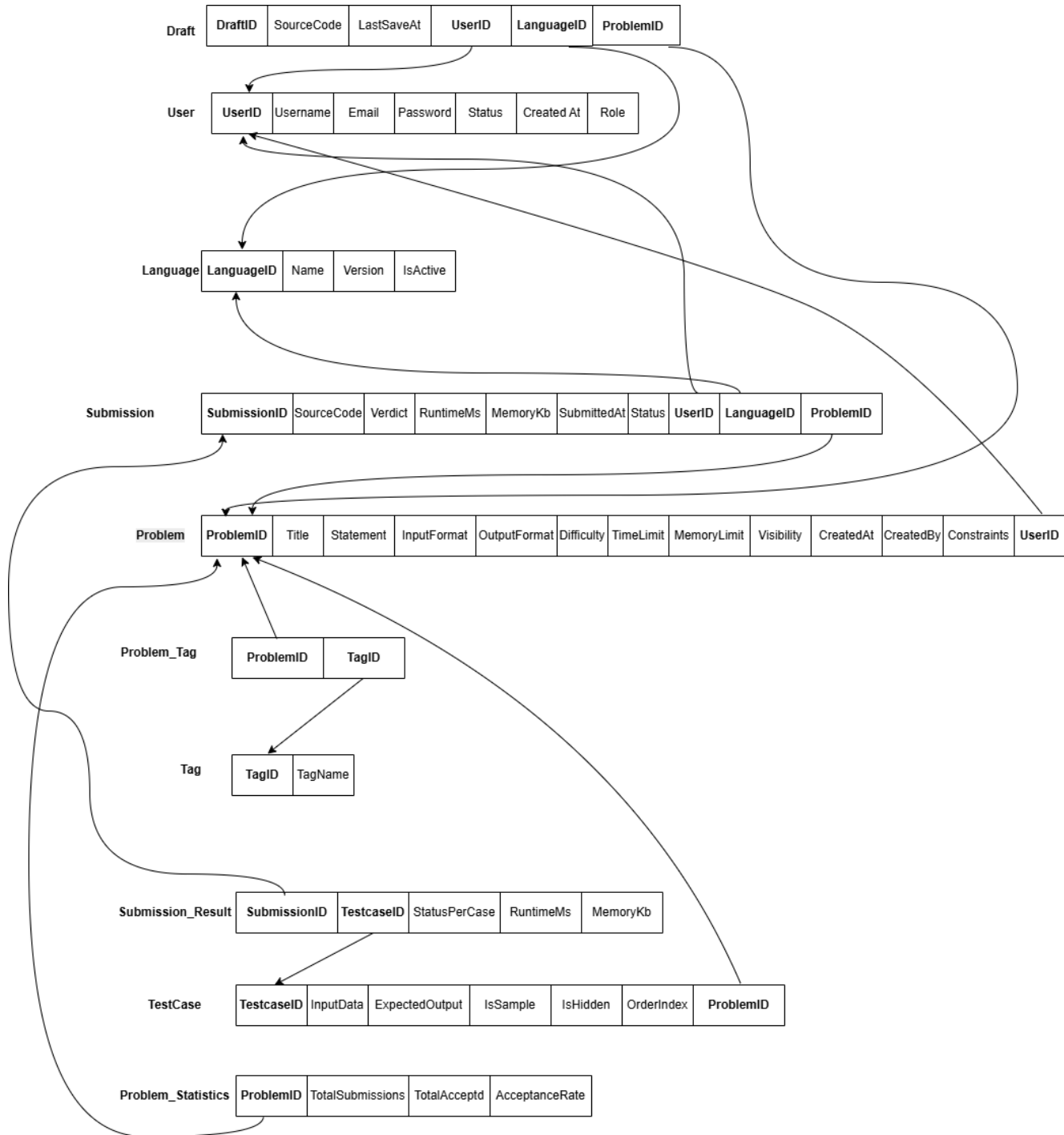
Seq	Property	Modifier	Constraint	Description
1	userId	private	Primary Key, Foreign Key	Associated user ID
2	problemsSolved	private	Integer, ≥ 0	Total problems solved
3	totalSubmissions	private	Integer, ≥ 0	Total submissions made
4	acceptedSubmissions	private	Integer, ≥ 0	Number of accepted submissions
5	acceptanceRate	private	Float, 0-100	Acceptance rate percentage
6	easyCount	private	Integer, ≥ 0	Easy problems solved
7	mediumCount	private	Integer, ≥ 0	Medium problems solved
8	hardCount	private	Integer, ≥ 0	Hard problems solved
9	ranking	private	Integer, > 0	Global user ranking
10	lastUpdated	private	Timestamp	Last statistics update

- Operations:

Seq	Operation	Modifier	Constraint	Description
1	updateStats(submissionResult)	public	Returns void	Updates stats after submission
2	getStats(userId)	public static	Returns UserStatistics	Retrieves user statistics
3	calculateAcceptanceRate()	private	Returns float	Calculates acceptance rate
4	updateRanking()	private	Returns void	Recalculates user ranking
5	incrementProblemsSolved(difficulty)	private	Returns void	Increments solved count

4. Data Design

4.1. Data Diagram



4.2. Data Specification

In this section, we describe the logical data model of the system and provide detailed specifications of all database tables used for data storage. For each table, the corresponding data columns are listed together with their attribute names, data types, value constraints, key constraints (primary keys and foreign keys), as well as a short explanation of the attribute's meaning and purpose within the system.

This specification serves as the foundation for implementing the physical database schema in the chosen Database Management System (DBMS) and ensures that the data structure consistently supports the system's functional and non-functional requirements

4.2.1. Table: USER

Description: Store information about all system users (students, problem setters, administrators)

Column	Data Type	Constraints	Description
UserID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier of the user.
Username	VARCHAR(50)	UNIQUE, NOT NULL	Login name of the user.
Email	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for

			registration and contact.
Password	VARCHAR(255)	NOT NULL	Encrypted password of the user.
Role	ENUM('student','setter','admin')	NOT NULL	Role of the user in the system.
Status	ENUM('active','banned','inactive')	NOT NULL DEFAULT 'active'	Account status.
CreatedAt	DATETIME	NOT NULL	Account creation timestamp.

4.2.2. Table: PROBLEM

Description: Store programming problems available in the system

Column	Data Type	Constraints	Description
ProblemID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier of the problem.
Title	VARCHAR(150)	NOT NULL	Title of the problem.
Statement	TEXT	NOT NULL	Full description of the problem.
InputFormat	TEXT	NOT NULL	Description of the input format.
OutputFormat	TEXT	NOT NULL	Description of the required output format.
Constraints	TEXT	NULL	Problem constraints (limits on

			input size, ...).
Difficulty	ENUM('easy','medium','hard')	NOT NULL	Difficulty level of the problem.
TimeLimit	INT	NOT NULL	Time limit per test case (in milliseconds)
MemoryLimit	INT	NOT NULL	Memory limit per test case (in kilobytes)
Visibility	ENUM('public','private','hidden')	NOT NULL DEFAULT 'public'	Visibility of the problem
CreatedAt	DATETIME	NOT NULL	Problem creation timestamp.
CreatedBy	INT	NOT NULL, FK → USER(userID)	User who created the problem (setter or admin)

4.2.3. Table: TAG

Description: Store tags used to categorize problems

Column	Data Type	Constraints	Description
TagID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier of the tag.
TagName	VARCHAR(50)	UNIQUE, NOT NULL	Name of the tag (for example: Array, DP).

4.2.4. Table: PROBLEM_TAG

Description: Link table for many-to-many relationship between PROBLEM and TAG

Column	Data Type	Constraints	Description
ProblemID	INT	PK, NOT NULL, FK → PROBLEM(ProblemID)	Linked problem
TagID	INT	PK, NOT NULL, FK → TAG(TagID)	Linked tag

4.2.5. Table: TEST_CASE

Description: Store test cases used to judge each problem

Column	Data Type	Constraints	Description
TestCaseID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier of the test case.
ProblemID	INT	NOT NULL, FK → PROBLEM(Problem ID)	Problem that this test case belongs to.
InputData	TEXT	NOT NULL	Input data for the test case.
ExpectedOutput	TEXT	NOT NULL	Expected correct output for the test case.
IsSample	BOOLEAN	NOT NULL DEFAULT 0	TRUE if this is a sample test shown to the user.
IsHidden	BOOLEAN	NOT NULL DEFAULT 1	TRUE if this test is hidden (used for judging).
OrderIndex	INT	NOT NULL	Order of the test case within the problem.

4.2.6. Table: LANGUAGE

Description: Store programming languages supported by the system

Column	Data Type	Constraints	Description
LanguageID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier of the language.
Name	VARCHAR(50)	NOT NULL	Language name (For example, C++, Python, Java).
Version	VARCHAR(20)	NULL	Version of the language (For example, C++17)
IsActive	BOOLEAN	NOT NULL DEFAULT 1	Whether the language is currently enabled.

4.2.7. Table: SUBMISSION

Description: Store each submission made by users for problems.

Column	Data Type	Constraints	Description
SubmissionID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier of the submission.

UserID	INT	NOT NULL, FK → USER(UserID)	User who made the submission.
ProblemID	INT	NOT NULL, FK → PROBLEM(ProblemID)	Problem being submitted.
LanguageID	INT	NOT NULL, FK → LANGUAGE(LanguageID)	Programming language used.
SourceCode	MEDIUMTEXT	NOT NULL	Submitted source code.
Verdict	ENUM('Accepted','Wrong Answer','Time Limit Exceeded','Memory Limit Exceeded','Runtime Error','Compilation Error','System Error')	NULL	Final result after judging.
RuntimeMs	INT	NULL	Total execution time across tests (milliseconds).

MemoryKb	INT	NULL	Peak memory usage (kilobytes).
Status	ENUM('Pending','Queuing','Judging','Done','Error')	NOT NULL DEFAULT 'Pending'	Processing status of the submission.
SubmittedAt	DATETIME	NOT NULL	Time when the submission was created.

4.2.8. Table: SUBMISSION_RESULT

Description: Store detailed results of each submission on each test case

Column	Data Type	Constraints	Description
SubmissionID	INT	PK, NOT NULL, FK → SUBMISSION(Submis sionID)	Related submission.
TestCaseID	INT	PK, NOT NULL, FK → TEST_CASE(TestCas eID)	Related test case.
StatusPerCase	ENUM('Passed','Failed','Time Limit Exceeded','Memory Limit')	NOT NULL	Result of this submission on the test case.

	Exceeded','Runtime Error')		
RuntimeMs	INT	NULL	Execution time on this test case (milliseconds).
MemoryKb	INT	NULL	Memory usage on this test case (kilobytes).

4.2.9Table: DRAFT

Description: Store temporary draft code that users save but do not submit yet

Column	Data Type	Constraints	Description
DraftID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier of the draft.
UserID	INT	NOT NULL, FK → USER(UserID)	User who created the draft.
ProblemID	INT	NOT NULL, FK → PROBLEM(ProblemID)	Problem related to the draft.
LanguageID	INT	NOT NULL, FK → LANGUAGE(LanguageID)	Language used in the draft.

SourceCode	MEDIUMTEXT	NOT NULL	Draft source code.
LastSavedAt	DATETIME	NOT NULL	Last time the draft was saved.

4.2.10 Table: **PROBLEM_STATISTICS**

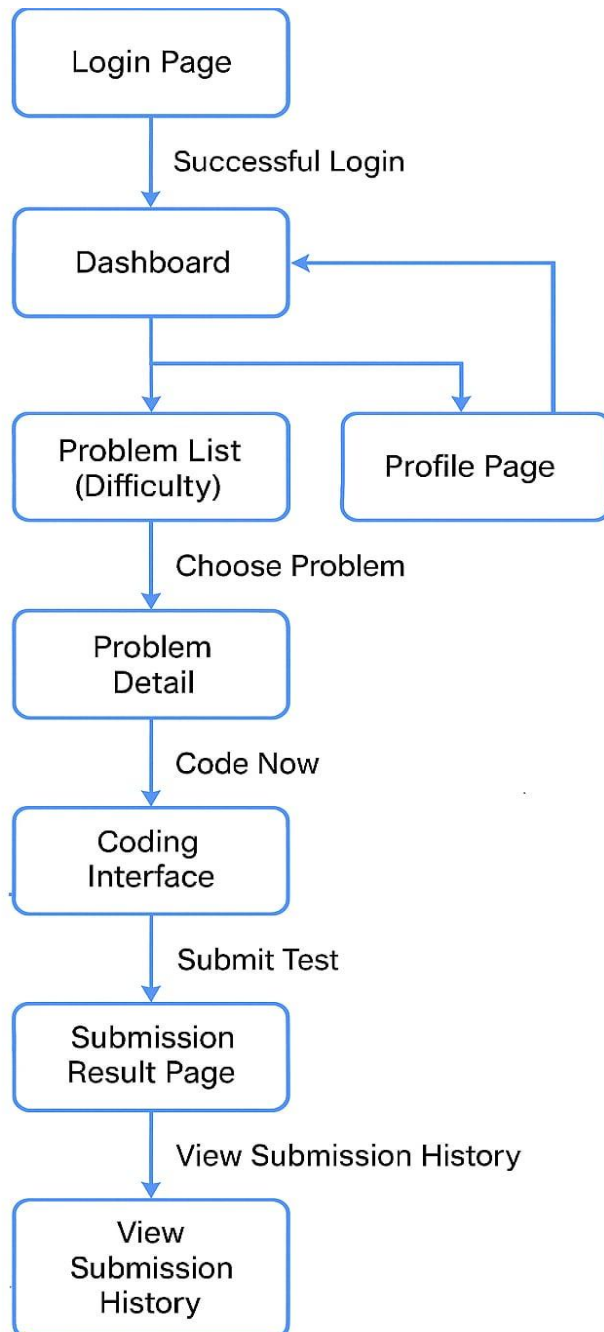
Description: Store aggregated statistics for each problem

Column	Data Type	Constraints	Description
ProblemID	INT	PK, NOT NULL, FK → PROBLEM(ProblemID)	Problem to which the statistics belong
TotalSubmissions	INT	NOT NULL DEFAULT 0	Total number of submissions for the problem.
TotalAccepted	INT	NOT NULL DEFAULT 0	Total number of accepted submissions.
AcceptanceRate	DECIMAL(5,2)	NULL	Acceptance rate (percentage).

5. User Interface & User Experience Design

5.1. Screen Diagram:

Screen diagram:



List of screens:

Seq	Screen Name	Description
1	Login Page	Entry page for user authentication via email/password or social login
2	Dashboard	Displays recent tests, progress bars, announcements, and user statistics
3	Problems Page	Shows all categories of problems and supports search + sorting
4	Problem List / Category Detail	Shows problem lists for selected category sorted by difficulty
5	Problem Detail	Shows learning objectives, examples, constraints, and “Code Now”
6	Coding Interface	Editor, console, language selector, run/submit controls
7	Submission Result	Displays execution/compile results and submission history

5.2. Screen Specifications:

5.2.1 Screen A - Login Page

Presentation format:

The login page features:

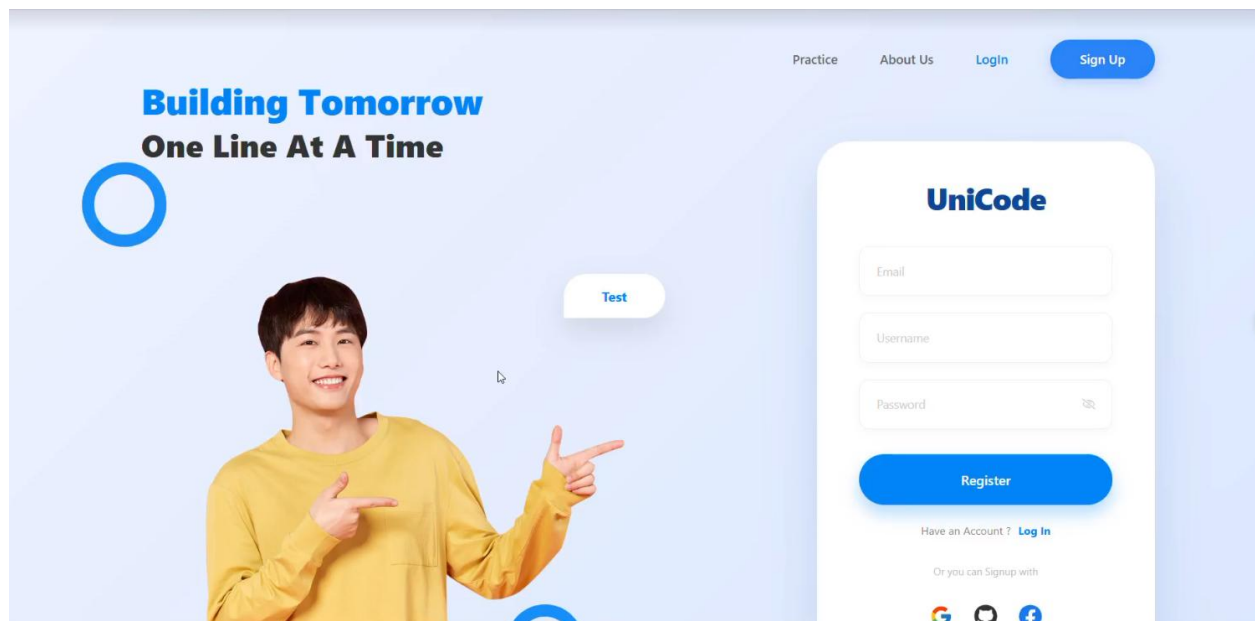
- A two-column layout:
 - Left: Motivational graphics
 - Right: Login card with rounded corners and soft-gradient background.
- Form Fields
 - Email input
 - Password input (with visibility toggle)
- Primary Button: Login
- Secondary Link: “Forgot Password?”
- Sign Up Navigation
- Third-Party Login Options: Google, GitHub, Facebook
- Responsive UI with centered card on smaller screens.

Event handling:

Event	Description
Login button click	Validates user credentials → On success, redirects to Dashboard; on failure, displays error.

Password visibility toggle	Switches input type between password and text.
Sign Up link	Navigates to the registration page.
Forgot Password	Opens password recovery workflow.
Social login icons	Invokes OAuth sign-in process.

Design image:



5.2.2 Screen B - Dashboard

Presentation format:

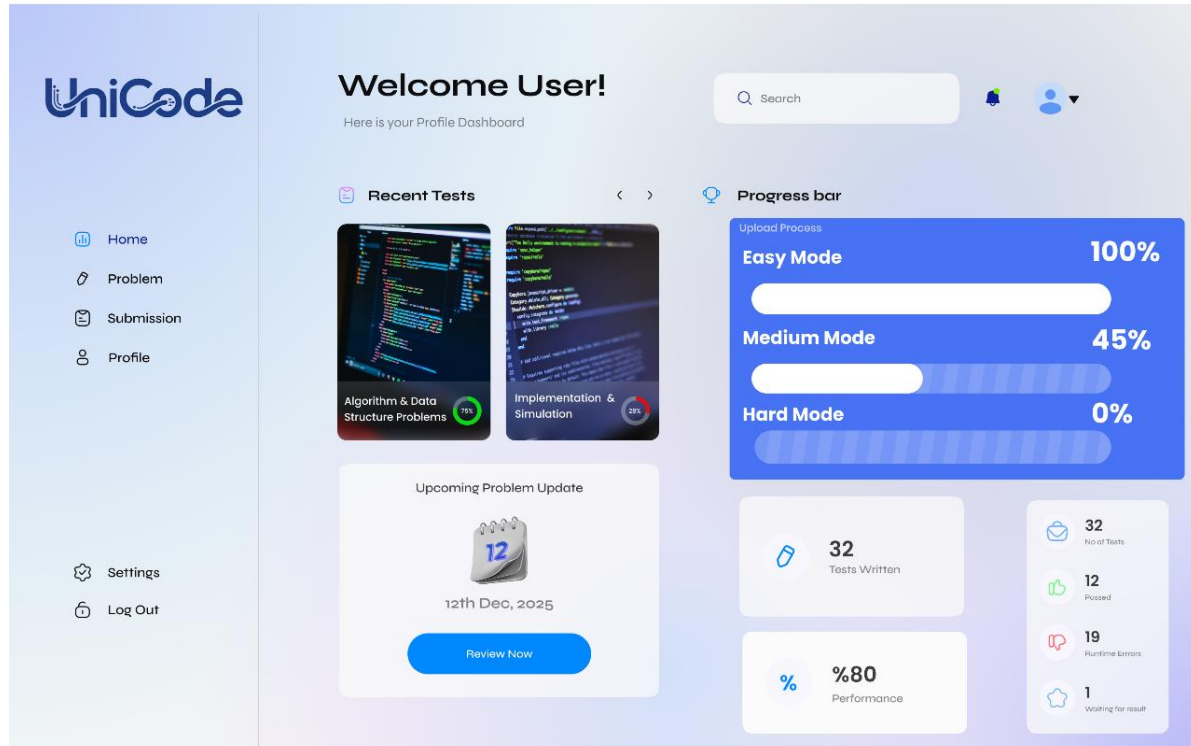
- Left Sidebar Menu: Home, Problems, Submission, Profile, Settings, Logout.

- Header: Search bar, notification icon, profile dropdown.
- Sections:
 - *Recent Tests*: Scrollable cards with problem name and completion percentage.
 - *Progress Bar Section*: Three difficulty progress metrics (Easy, Medium, Hard).
 - *Upcoming Problem Update*: Announcement card with date and call-to-action button.
 - *User Stats*: Number of tests written, performance %, passes, runtime errors, etc.

Event handling:

Event	Description
Select sidebar menu item	Navigates to the respective screen.
Problem card click	Open Problem Detail screen.
Progress bar hover	Tooltip shows additional progress statistics.
Review Now button	Opens details related to upcoming problem update.
User icon dropdown	Change profile settings or logout.

Design Image:



5.2.3. Screen C - Problem List (Choosing a Problem)

Presentation format:

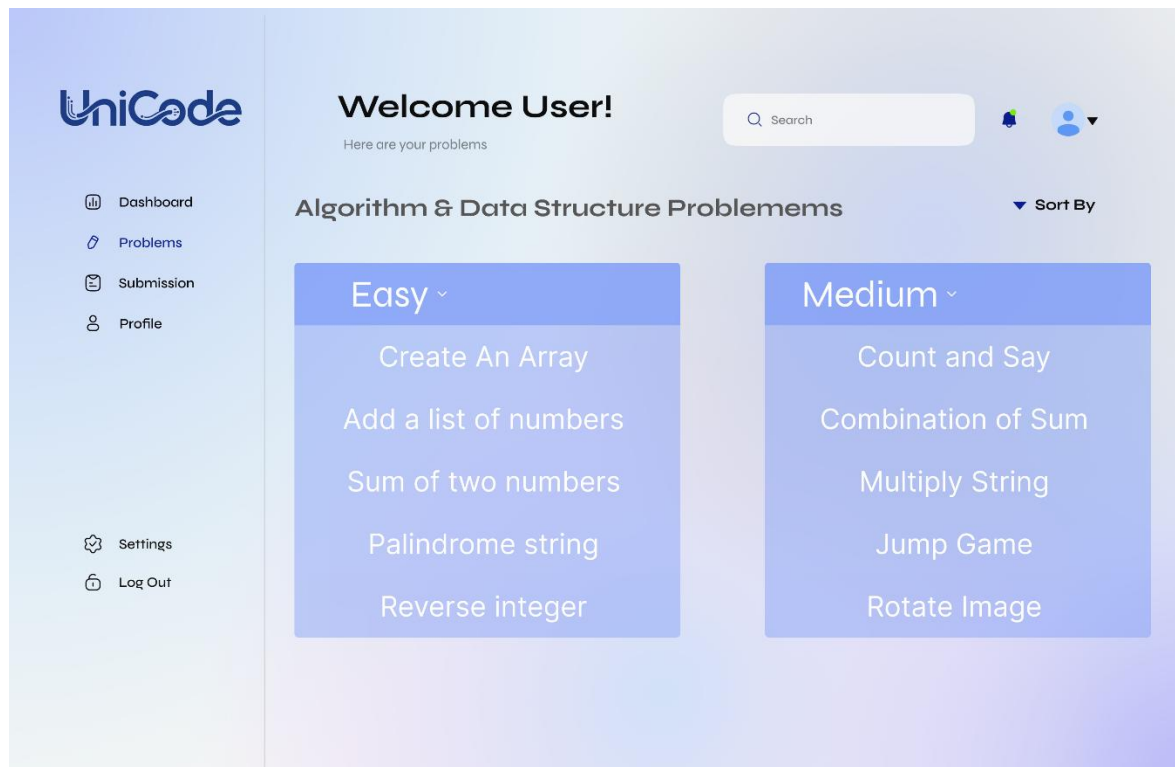
Shows a list of problems divided by difficulty tabs:

- Tabs: Easy, Medium, Hard (highlighted color bar).
- For each difficulty:
 - List of problem titles (clickable)
 - Consistent box layout with subtle borders
- Global elements:
 - Search bar
 - Sort By dropdown

Event handling:

Event	Description
Selecting difficulty tab	Refreshes visible problem list.
Clicking a problem	Open Problem Detail screen.
Search input	Filters problem names.
Sorting list	Re-arranges according to selected rules.

Design image:



5.2.4. Screen D - Problem Detail Page

Presentation format:

- Top section:
 - Problem title
 - Acceptance rate, total attempts
 - Difficulty tag (color-coded: Easy = green, Medium = yellow, Hard = red)
 - Status tag (Solved / Unsolved)
- Learning Objectives: Bullet points describing expected skills.
- Problem Summary: Short explanation of the problem.
- Examples (Example 1, Example 2) in blue-bordered boxes.
- Constraints listing allowed ranges.
- “Code Now” button leading to the coding interface.

Event handling:

Event	Description
Click “Code Now”	Opens Coding Interface with problem preloaded.
Scroll interaction	Sticky sidebar remains visible to maintain navigation.

Design image:

UniCode **Sort an array** Medium Unsolved

Acceptance Rate: 63% Total Attempts: 18,240 Solved by: 11,502 users

Learning Objectives

After completing this problem, you will practice:

- Implementing classical sorting algorithms.
- Understanding merge sort / quicksort logic.
- Achieving $O(n \log n)$ time complexity.
- Handling large arrays efficiently.
- Improving clean code skills with recursive or iterative algorithms.

Problem Summary

You are given an integer array, and you must sort it in ascending order and return the sorted array. You may use any sorting algorithm, but an **efficient solution is recommended**.

EXAMPLE 1

Input: nums = [5,2,3,1]	Output: [1,2,3,5]
-------------------------	-------------------

Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5)

EXAMPLE 2

Input: nums = [5,1,1,2,0,0]	Output: [0,0,1,1,2,5]
-----------------------------	-----------------------

Explanation: Note that the values of nums are not necessarily unique.

Constraints

- $1 \leq \text{nums.length} \leq 5 \times 10^4$
- $-5 \times 10^4 \leq \text{nums}[i] \leq 5 \times 10^4$

[CODE NOW](#)

5.2.5. Screen E - Coding Interface

Presentation format:

This is the core practice environment consisting of:

- Problem Description Panel (top section)
- Programming Panel (bottom left):
 - Language dropdown (C++, Python, Java, etc.)
 - Code editor with line numbers & dark theme
- Console Panel (right side):
 - Sample test inputs
 - Custom input field

- Run Button (bottom left)
- Submit Test Button (bottom right)
- Timer icon on top right

Event handling:

Event	Description
Language selection	Reloads editor with boilerplate code for selected language.
Run button	Compiles locally and displays console output.
Submit button	Sends code to judge → redirects to Submission Result page.
Entering custom input	Overrides sample test and uses user-provided input.

Design image:

UniCode

00:20:30

Programming

Sort An Array

You are given an integer array, and you must sort it in ascending order and return the sorted array. You may use any sorting algorithm, but an **efficient solution is recommended**.

Example 1:
Input: nums = [5,2,3,1]
Output: [1,2,3,5]
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:
Input: nums = [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
Explanation: Note that the values of nums are not necessarily unique.

Constraints:

- $1 \leq \text{nums.length} \leq 5 \times 10^4$
- $-5 \times 10^4 \leq \text{nums}[i] \leq 5 \times 10^4$

C++

```
1 #include <iostream>
2 Pair<int, int> SortAnArray(Vector<int>& Arr, int N, int K)
3 {
4     // Write Your Code Here
5 }
```

Console

Sample Custom

Write Your Input Cases Here Like 5231

Run

Submit Test

5.2.6. Screen F - Submission Result Page

Presentation format:

Shows status after code submission:

- Result Panel (red for compile/runtime errors, green for success)
- Parsed error message with line number indicators
- Test execution details: runtime, memory usage
- Navigation:
 - Previous Test
 - Next Test
- Submission History Table:
 - Status
 - Language
 - Runtime
 - Memory

Event handling:

Event	Description
View History button	Expands complete submission list.
Prev/Next Test	Browse test cases executed.
Click submission row	Loads details of chosen submission.

Design image:

